Module 3: Parsing

Parser Generators

Why write a parser when you can generate one?

0.

Assignment Review

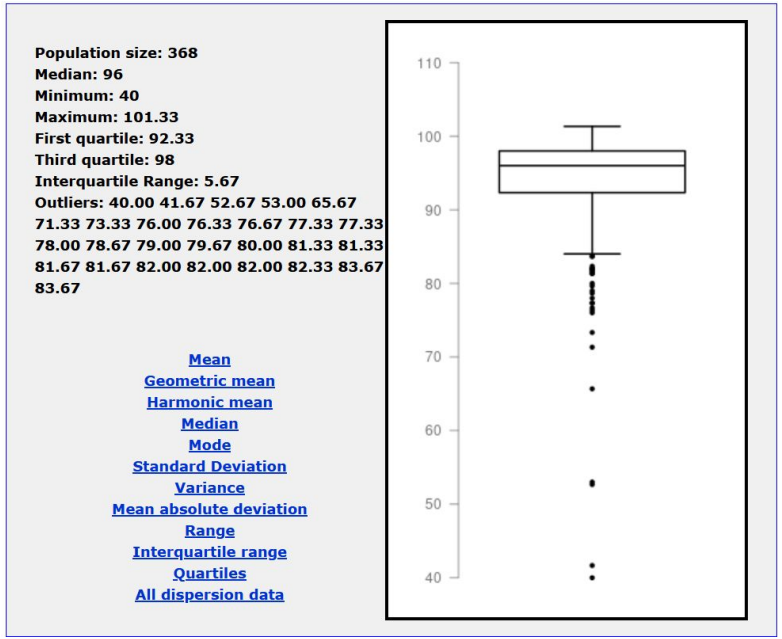
RegexTesting + Module 2 Quiz

RegexTesting (FINAL)

Average 93.9%, Median 96.0%

Grades are as expected, comparable to test submission results as well.

- Coverage was the big issue; many wrote only 10 tests total.
- Captialized exponent 1E10?
- RegexFileFilter overall fine.
  - Many tried to use File IO operations, e.g. exists?
  - listJavaFiles is just dir.listFiles.



1.

Parser Generators

Why write a parser when you can generate one?

Why not use the grammar to parse?

1. Not all grammars are the same (lookahead, ambiguity)
2. Libraries have tradeoffs (dependencies, error handling)

Parser Generators

PEG Parsing

PEG is **parsing expression grammar**, which is built specifically for parsing.

- PEGs remove ambiguity by adding parsing decisions to the grammar.
- Primarily “first come first parsed”:
  - `expr ::= 'NIL' / ident`
- Also allows “not” and “and” logic.
- Roughly, corresponds to recursive descent with backtracking.

EBNF* Parsing

Some tools support “real” EBNF (or other) grammars, applying additional constraints.

- Each tool has it’s own “style” of grammars that it’s focused on (LL, LR, etc.).
  - May support others to a lesser extent, thus being a bit more versatile/flexible.
- Ambiguity is handled via precedence rules, heuristics, and/or compile/runtime detection.
- Typically these “larger” tools offer a lot of additional features for parsing as well.

Tradeoffs

Advantages

- Definition of correctness comes directly from the grammar itself.
- In theory, speeds up development. In practice, changes aren't so quick.
  - Generally best when trying to parse an existing language.
- Typically more efficient than a naive handwritten solution.
- Most libraries have custom features adding additional language power.

Disadvantages

- Relies on a library/dependency, which *always* has it's own series of quirks.
  - *Might* simply result in lost time learning, understanding, and debugging the tool.
  - Advanced library features add additional complexity to a "simple" grammar.
  - May need a rewrite if you outgrow the tool.
- Ambiguities can be harder to see/identify, as the parsing logic is effectively hidden.
- Error handling tends to be *significantly* worse, especially as more of an "end user" product.

PEG Parsing

https://peggyjs.org/online.html

EBNF* Parsing

http://lab.antlr.org

2.

Parser Combinators

Why write a grammar when you can make the parser?

Parser Combinator

Definitions

A parser combinator builds a parser by chaining together parser functions.

- Higher-order functions allow combining parser operations.
  - `pFirst("NIL", pVar)`
  - `pSeq(pExpr, sep=",")`
- Acts like a “mini-DSL” for building parsers that reads like a grammar.

Tradeoffs

- Similar to a parser generator, but more versatile with less library lock-in.
  - Likewise, similar tradeoffs.
- Allows building your own helpers to match the style of the language being parsed.
  - Question: Can’t we do the same thing with a “regular” recursive descent parser?
    - Yes, but trickier to compose effectively.
- Often messier around ambiguity, lookahead, and error handling (relies on backtracking).

Module 3: Parsing

Parser Generators

Why write a parser when you can generate one?

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals